

基于 SpacemiT K1 Muse Pi Pro 的 AI Agent 智能快递分拣中枢系统

摘要

随着快递业务持续向末端延伸，社区驿站、校园快递点等场景普遍面临人工成本居高不下、包裹信息滞后、高峰期拥堵等运营压力，无人化改造需求日益迫切。本系统以进迭时空 SpacemiT K1 RISC-V 开发板为核心主控，构建从包裹入站到用户取走的全自动化智能管理平台。

硬件层面，系统采用团队自主设计 STM32F411RET6 下位机控制板与 K1 供电模块，通过步进电机滑台与机械夹爪实现包裹的自动分拣与存取，多路出口支持并发出库。视觉层面，摄像头图像经 K1 板载 RVV 向量指令集加速预处理后，送入板载的 NPU 量化 YOLOv8n 模型执行实时目标检测，结合 PID 闭环控制引导机械臂精确定位，视觉推理与运动控制全程在边缘侧完成，无云端算力依赖。

交互层面，系统提供四种取件方式：触摸终端手机尾号自助取件、自然语言 AI 对话取件、手机 App 一键远程取件、飞书群聊无接触取件。AI 取件以 DeepSeek 大模型为推理核心，通过结构化 JSON 协议严格约束模型输出范围，AI 负责意图理解，确定性代码负责硬件执行，兼顾智能性与可靠性。四种入口共用同一套工具实现，扩展新入口无需修改已有逻辑。

数据层面，云端部署 PHP 接口与 MySQL 数据库，包裹全生命周期状态唯一存储于云端，Qt 触摸终端、Web 管理后台、Android App 实时读取，保证多端数据强一致性。App 取件请求经数据库消息队列异步传递至 K1 硬件侧消费，两端不直连，断网恢复后自动续处理。入库异常由系统自动上报工单，出库异常支持用户通过 AI 对话上报，管理员可在 Web 与 App 端查看、处置与导出，形成完整的异常追溯链路。

系统适用于社区驿站、校园快递点、写字楼收发室等无人值守场景，格口规模、取件入口与站点数量均可按需扩展，具备较强的实际部署可行性。本系统实现了感知、推理、控制、交互、数据五层的完整工程闭环，充分发

挥 RISC-V 边缘计算平台的异构算力优势，为末端快递站的无人智能化改造提供了一套可落地的完整解决方案。

摘要	1
第一部分 作品概述	5
1.1 功能与特性	5
1.2 应用领域	6
1.3 主要技术特点	7
1.4 主要性能指标	7
1.5 主要创新点	8
1.6 设计流程	8
第二部分 系统组成及功能说明	10
2.1 整体介绍	10
①系统的起点：感知层	10
②系统的执行核心：控制层	10
③系统的智能决策层：AI Agent	11
④系统的用户入口：多端应用层	12
⑤系统的数据基础：云端数据层	13
⑥各层之间的协作关系：	14
2.2 硬件系统介绍	15
2.2.1 硬件整体介绍	15
(1) 主控计算板——SpacemiT K1 Muse Pi Pro	15
(2) 下位机控制板与供电模块	16
(3) 执行机构	17
(4) 交互与显示设备	18
(5) 硬件子系统间的连接关系	18
2.2.2 机械设计介绍	19
(1) 整体机架结构	19
(2) 皮带输送机构	19
(3) 直线分拣执行机构	19
(4) 视觉采集与人机交互单元	19
2.2.3 电路各模块介绍	20
(1) 电源模块	20
(2) MCU 主控	21
(3) 外设接口与驱动模块	21
2.3 软件系统介绍	22
2.3.1 软件整体介绍	22
(1) K1 本地软件结构	23
(2) 云端软件结构	24
(3) App 软件结构	25
2.3.2 软件各模块介绍	25
(1) 扫码识别管线	25
(2) 运动控制模块	26
(3) 自然语言到硬件动作的转化路径	28
(4) 六表结构与包裹全生命周期数据设计	29
第三部分 完成情况 & 性能参数	30

3.1 整体介绍	30
3.2 工程成果	31
3.2.1 机械成果	31
3.2.2 电路成果	32
3.2.3 软件成果	34
3.3 特性成果	36
(1) 视觉感知性能	36
(2) AI Agent 取件	36
(3) 多端功能覆盖	37
(4) 数据管理	37
(5) 系统集成稳定性	37
第四部分 总结	38
4.1 可扩展之处	38
4.2 心得体会	38
第五部分 参考文献	40

第一部分 作品概述

1.1 功能与特性

末端快递站面临全天候驻人成本高、高峰期排队体验差等痛点。本系统以进迭时空 K1 开发板为核心主控，构建从包裹入站到用户取走的新时代 AI 全自动化管理平台，支持四类入口并发使用，大大缩减所需驻站人员数量。

（1）**入库端：**摄像头扫描快递 QR 码，识别单号、收件人手机号与物品信息，自动完成商品分类后实时上传云端数据库，进而驱动步进电机与机械夹爪将包裹精确存入对应格口，并将物品信息及时推送至站点大屏。

（2）**取件端：**提供三端交互方式——触摸端输入尾号自助取件、手机 App 一键远程取件、飞书群聊无接触取件，并且支持手机尾号/单号模糊查询，按彩色状态徽标区分在库与已取状态；三端均支持自然语言诉求实现 Agent 自动取件。

（3）**账号体系：**手机号注册登录，含验证码防刷；忘记密码可经验证后在线重置；管理员入口设 6 位 PIN 码二次鉴权；Web 网页端、App 端支持游客无登录直接查询在库包裹位置。

（4）**管理端：**Web 后台与 App 管理员端提供包裹全量管理、六维数据分析看板、异常工单处理、CSV 批量导入导出；App 支持 EAS Build 云端打包与 OTA 热更新，JS 层变更无需用户重新安装。



图 1-1 QT 端界面展示

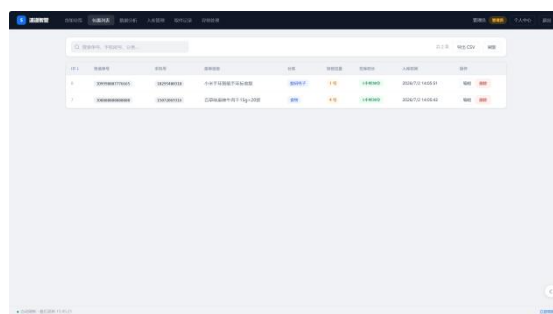


图 1-2 Web 端页面展示



图 1-3 APP 端界面展示



图 1-4 飞书端页面

1.2 应用领域

本系统定位于末端快递站点的无人智能化改造，核心解决以下四类问题：

（1）**无人自主化**：从包裹到站到用户取走，全程无需站点人员或仅需少数站点人员介入。摄像头扫码、机械臂分拣、格口分配、硬件出库均由系统自主完成，有效缩减人力成本，适用于无人值守的社区驿站、校园快递点等场景。

（2）**模糊交互化**：传统自助取件依赖精确输入，本系统通过 AI 大模型支持自然语言交互，用户只需用口语描述需求（“帮我取手机尾号 1234 的包裹”“10 分钟后我到，帮我准备一下”），系统自动理解并执行，有效降低老年群体等特殊用户的使用门槛。

（3）**数据可溯源化**：所有包裹从入库到取走的完整生命周期均记录于云端数据库，包含快递单号、物品信息、格口位置、入库时间、取件时间等字段。异常事件自动生成工单，可查、可追、可导出，满足站点日常运营与纠纷处理的数据管理需求。

（4）**效率提升化**：双路并发出库、App 远程预约取件（到站即取）、四种渠道同时在线，显著提升站点吞吐量与用户取件体验，减少高峰期排队

等候时间。

1.3 主要技术特点

(1) **采用 RISC-V 边缘计算平台：**以进迭时空 SpacemiT K1 Muse Pi Pro 为主控，利用板载 NPU 与 RVV 向量指令集本地完成 AI 推理与图像加速，无需依赖云端算力执行实时视觉任务。

①**YOLOv8n QDQ 量化 NPU 部署：**目标检测模型经 QDQ 格式量化后部署于 K1 使用 NPU 执行 INT8 推理，实现稳定 20+ FPS 的实时包裹追踪，置信度 0.95 以上。

②**RVV 向量指令加速图像预处理：**畸变校正 (cv2.remap) 利用 SpacemiT RVV HAL 实现向量化加速，较纯 Python 实现提速 14.4 倍，确保预处理不成为视觉管线瓶颈。

(2) **AI Agent 工程化驱动硬件：**以 DeepSeek 大模型为推理核心，通过 OpenClaw Agent 框架与自研硬件插件，将自然语言转化为结构化 JSON 动作序列，确定性代码负责硬件执行，兼顾智能性与可靠性。

(3) **多协议异构通信：**单一边缘节点同时运行 UART 串口自定义帧、串口屏协议、HC-42 蓝牙透传、MySQL 云端同步 (TCP)、HTTPS 云端 AI 对话 (DashScope API)、HTTP 局域网直连 (App)、HTTPS 设备注册 (K1 IP 动态上报) 共七种通信协议链路，实现异构设备统一调度。

(4) **多端统一后端：**触摸屏终端、Web 管理后台、Android App、飞书群聊四种入口共用同一套 PHP 接口与 MySQL 数据库，权限体系统一，数据实时同步。

1.4 主要性能指标

性能指标	实测数值	测试条件
图像畸变校正耗时 (RVV 加速)	3.36ms/帧 (等效 297FPS)	800×480, K1 板载
图像畸变校正耗时 (Python 基准)	48.33ms/帧	同分辨率, 纯 numpy
RVV 向量加速比	14.4×	SpacemiT RVV HAL ver 0.0.1

YOLOv8n NPU 推理帧率	稳定 20+FPS	QDQ 量化, INT8, K1 NPU
目标检测置信度	>0.95	实际场景稳定值
视觉闭环对齐精度 (均值)	1.5mm	50 次入库实测
视觉闭环对齐精度 (最大值)	3.3mm	50 次入库实测
追踪收敛耗时 (典型值)	~274ms	4—6 次 PID 迭代
电机控制指令频率	11—13Hz	PID 闭环控制周期
端到端取件延迟	3.7s	APP 端发送取件请求
系统连续运行时长	17h	实验室下测得最长运行时间

1.5 主要创新点

(1) **异构计算分工**: NPU 专供视觉推理, RVV 加速预处理, 语言推理上云, 三者并行互不干扰。

(2) **JSON 约束控制**: 用 JSON 严格约束 LLM 输出, 实现 AI 决策与代码执行分离, 标准工具多端共用。

(3) **视觉闭环端侧化**: 图像采集至电机控制全链路本地部署, 精度 1.5mm, 无云端依赖。

(4) **消息队列解耦**: 以数据表为异步队列解耦多端, 支持断网恢复自动续处理, 防并发冲突。

(5) **异常工单闭环**: 出入库异常自动入库, 支持多端查看处置与导出, 链路完整可追溯。

1.6 设计流程

本项目采用“硬件先行、软件并进、逐层联调”策略, 分四阶段推进:

(1) **需求与选型**: 明确快递站自动化需求, 选定 K1 为主控及各执行模块, 设计整体架构。

(2) **硬件搭建**: 自主设计下位机 PCB 与供电模块并打板验证, 加工搭建原创机械结构, 联调串口协议。

(3) **软件并行**：视觉感知、运动控制、AI Agent 工具库、云端、终端及 App 等多模块并行开发与独立测试。

(6) **联调优化**：完成扫码到出库全链路集成联调，优化 RVV 及 PID 参数，开展多端压力与异常测试。

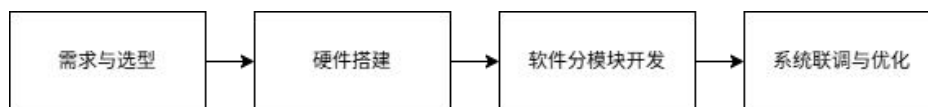


图 1-5 系统设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

本系统以进迭时空 SpacemiT K1 Muse Pi Pro 为核心边缘计算节点，围绕末端快递站“无人值守、全流程自动化”的目标，将感知、推理、控制、交互、数据五个维度整合为一套完整的自动化管理平台。整个系统在物理上分为两个运行域——K1 本地边缘节点与阿里云 ECS 云端服务器，两者通过 HTTPS REST 接口持续互联；在逻辑上则分为五个功能层，各层职责清晰、接口标准化，层内变动不影响上下层的正常运行。

①系统的起点：感知层

包裹进入站点后，系统通过连接于 K1 USB 端口的摄像头完成对物理世界的感知。感知层由两条并行管线构成，分别承担不同的任务。

第一条是扫码识别管线，由 `scan_json.py` 实现。摄像头采集的每一帧图像首先经过 RVV HAL 向量加速的畸变校正，以消除镜头畸变对二维码识别的干扰，随后交由 `pyzbar` 库完成二维码的识别，解析出快递单号、收件人手机号与物品信息，再通过关键词字典将物品归入日常用品、衣服、食物、保密发货四类。识别完成后，脚本向云端发起接口请求，查询当前空闲格口并完成入库记录的写入。这条管线的输出是一条已落库的包裹记录，包含包裹身份与格口分配两项关键信息。

第二条是视觉闭环追踪管线，由 `vision.py` 与 `tracker.py` 协同实现。同样经过 RVV 畸变校正后，帧数据被送入部署于 K1 板载 NPU 的 YOLOv8n 量化模型，输出当前帧中包裹目标的包围盒坐标，再经坐标系变换后输入 PID 控制器，计算出驱动步进电机跟随目标所需的脉冲指令。这条管线以 11—13Hz 的频率持续输出控制指令，直至目标对齐精度收敛至 1.5mm 均值范围内。

这两条管线在时序上是分工协作的关系：扫码管线负责在包裹到达前完成身份识别与格口分配，将结果写入数据库；追踪管线负责在包裹进入识别区后引导机械臂精确定位，从数据库读取前者写入的格口分配信息，完成最终的入库动作。两者通过 MySQL 数据库完成信息传递，耦合度极低。

②系统的执行核心：控制层

感知层的输出只是坐标与分配信息，将其转化为实际物理动作的是运动控制层，由 `pack_in.py`（入库）与 `pack_out_ai.py`（出库）两个脚本实现，底层通过两条独立的串口链路分别控制步进电机和机械夹爪。

步进电机通过 `/dev/ttyUSB0` 与控制器通信，采用自定义二进制帧协议，每次运动前先发送清零帧使编码器复位，再发送绝对位置运动帧，保证重复定位精度。格口编号到脉冲数的映射由配置表决定，四个预设位置覆盖全部 1—8 号格口。机械夹爪通过 `/dev/ttyACM0` 与自研 STM32F411RET6 下位机通信，采用字符串指令格式，每条指令后强制延时 100ms 并调用 `flush()`，以防止 MCU 串口缓冲区因 K1 多任务并发产生时序抖动时发生漏指令。上电时执行三次缓冲区清空序列，确保指令时序从干净状态开始。

入库流程由视觉识别信号触发，包裹被识别后，`pack_in.py` 读取数据库中 `scan_json.py` 已写入的格口分配，启动视觉闭环追踪引导机械臂对准传送带上的包裹，追踪收敛后执行抓取，随后驱动步进电机将包裹移至目标格口并放入。若追踪超时，脚本自动调用 `hardware_tools.py` 创建入库异常工单并写入数据库，不阻塞后续包裹的处理。

出库流程由上层应用层触发，接收到格口编号后，`pack_out_ai.py` 驱动步进电机移至目标格口，机械臂执行抓取，再将包裹搬运至出口区放下，步进电机返回准备位，最后串口关闭、资源释放。奇数格口送至左出口区，偶数格口送至右出口区，两路电机地址不同，可实现双路并发出库。

③系统的智能决策层：AI Agent

控制层的接口是确定性的——给定格口编号，硬件执行动作。而将用户的自然语言转化为格口编号这一步，由 AI Agent 层负责完成。

AI Agent 层以 DeepSeek 大模型为推理核心，通过 OpenClaw Agent 框架调度，硬件功能封装为四个标准工具：查询包裹（`hw_query_packages`）、驿站忙闲查询（`hw_get_busyness`）、创建异常工单（`hw_create_anomaly`）、预约取件（`hw_prepare_pickup`）。这四个工具的实现统一在 `hardware_tools.py` 中，供飞书、App、Qt 触摸终端三端共用，新增接入端无需修改工具实现本身。

AI 与控制层的边界通过 JSON 协议划定：LLM 只负责意图理解与参数

提取，输出严格约束为包含 message 与 actions 数组的 JSON 结构；确定性代码解析 JSON 后逐一执行出库动作，硬件执行路径完全绕开 LLM 的不确定性。上下文注入机制确保 LLM 在收到用户消息时已拥有包裹格口、单号等全部技术细节，无需向用户二次询问。预约取件场景下，倒计时运行于独立的 detach 进程，用户收到确认回复即可挂断对话，到时自动触发出库并推送大屏通知，AI 响应与硬件执行完全异步解耦。

三端的调用路径存在差异。Qt 终端通过 CLI 调用 OpenClaw agent，走本地进程通信；飞书通过 OpenClaw 飞书渠道接入 DeepSeek，再由插件调用 hardware_tools.py；App 则绕过 OpenClaw Node.js 进程，直接向 K1 本地的 chat_server.py(HTTP 8765 端口)发起请求，chat_server.py 直连 DeepSeek API，响应时间从 7—15 秒降至 3—8 秒。chat_server.py 由 systemd 用户服务管理，K1 开机自动启动；K1 的局域网 IP 由 chat_server.py 每 5 分钟向云端 register_device.php 续报，TTL 10 分钟，App 通过查询该接口动态获取 K1 地址，适应热点切换场景，无需手动配置 IP。

④系统的用户入口：多端应用层

应用层对外提供四种取件与管理入口，共用同一套云端数据层，端侧不持有任何业务状态副本。

Qt 5 触摸终端是本地最核心的交互界面，运行于 K1 连接的 800×480 IPS 触摸屏。它既是用户的本地自助取件终端，也是 K1 所有后台子进程的启动与管理控制台。Qt 主进程通过 QProcess::startDetached 异步调用 motor.py 执行出库，通过 QNetworkAccessManager 的异步接口访问云端数据，UI 主线程任何时候都不阻塞等待硬件或网络操作。系统设置模块管理三个长生命周期子进程（摄像头扫码、大屏推送、包裹入库），停止时先向 Python 进程发送 SIGINT 触发 finally 中的硬件归位代码，确保电机与机械臂安全复位后再分级 terminate/kill 清理，避免执行机构停在中间位置。

React Native App 是移动端入口，支持 Android 平台，通过 EAS Build 云端构建，无需本地 SDK 或 Xcode 环境。JS 层变更通过 OTA 热更新推送，已安装的 App 下次启动自动更新，无需用户重新安装。一键取件的完整链路跨越三个执行环境：App 写入 pending 请求到云端数据库，K1 本地的

pickup_poller.py 每 3 秒轮询消费，完成出库后回写 done/failed 状态，App 轮询到结果后展示对应动画弹窗。两端不直连，完全通过数据库中的状态字段交换信息，断网恢复后轮询自动续处理，天然具备断线恢复能力。

Web 管理后台部署于阿里云 ECS，以纯原生前端（零框架、零图表库）实现，通过六维数据分析看板、异常工单处理、CSV 批量导入导出等功能支撑站点日常运营。数据每 3 秒自动刷新，页面隐藏时暂停轮询，恢复焦点后立即更新。

飞书群聊通过 OpenClaw 飞书渠道接入，是四种入口中唯一不依赖专用客户端的方式，用户在日常沟通群中即可完成查件、上报异常、预约取件等操作，进一步降低使用门槛。

淘晶驰串口屏是站点大屏，由 parcel_display.py 每 3 秒从数据库读取在库待取包裹信息，通过 `/dev/ttyS0` 以串口屏协议写入，HC-42 蓝牙模块作为串口透传桥接器，支持无线灵活部署，无需走线。

⑤系统的数据基础：云端数据层

所有端共用同一套云端数据层，部署于阿里云 ECS，由 26 个 PHP REST 接口与 6 张 MySQL 表构成。包裹全生命周期的权威状态只在 MySQL 中维护，Qt、Web、App 全部实时从云端读取，端侧不持有任何业务状态副本，保证多端数据强一致性。

shipments 表记录包裹从入库到取走的完整生命周期；pickup_requests 表充当 App 与硬件之间的异步消息队列，是实现多端取件联动的关键中间层；anomaly_reports 表统一承载入库异常与出库异常，管理员可在 Web 与 App 端查看、处置、导出，形成可追溯的完整异常链路；push_tokens 表存储 Expo 推送令牌，支持包裹到站时向用户 App 发送通知；user_tokens 表管理 Bearer Token 鉴权，每次登录生成新 token，支持 Web Session 与 Bearer Token 双模式统一鉴权，同一套接口无需维护双份实现；users 表以手机号为唯一登录账号，区分普通用户与管理员两种角色，权限体系贯穿全部端。

格口并发冲突通过三层防护解决：Python 端预查空闲格口作为第一层软过滤，PHP 接口执行 INSERT 前再次 SELECT 确认作为第二层原子校验，409 冲突时 Python 端重新选取格口作为第三层兜底重试，三道防线确保高并发入

库时包裹不丢失、格口不冲突。全部数据库操作使用 Prepared Statement 与 bind_param, 前端所有动态内容经 HTML 转义, 防范 SQL 注入与 XSS 攻击。

⑥各层之间的协作关系:

五层之间的信息流向是明确且单向的: 感知层采集物理信号, AI 推理层输出坐标与语义理解结果, 控制层将两者转化为硬件动作, 应用层承载用户交互与业务触发逻辑, 云端数据层提供持久化存储与多端同步。向上方向, 每一层只向上层提供标准化的输出——坐标数据、JSON 动作序列、REST 接口返回值; 向下方向, 每一层只通过标准协议向下层发出指令——串口帧、CLI 调用、HTTP 请求。这种分层解耦使得系统在联调阶段可以逐层验证: 先在 PC 上联调云端接口与 App, 再在 K1 上联调串口通信, 再完成视觉管线集成, 最后联调全链路, 任何一层出现问题都能快速定位。

K1 单节点在正常运行时同时承载七种通信协议链路, 这七种协议各自运行于独立的串口设备或网络接口, 互不干扰, 体现了 RISC-V 边缘节点的异构数据整合能力。Qt 通过 QProcess 与 QNetworkAccessManager 的异步机制管理全部子进程与网络请求, NPU 专用于视觉推理, CPU 主线程专用于 UI 事件响应, 高耗时操作均在子进程或异步回调中完成, 界面在任何情况下保持响应。

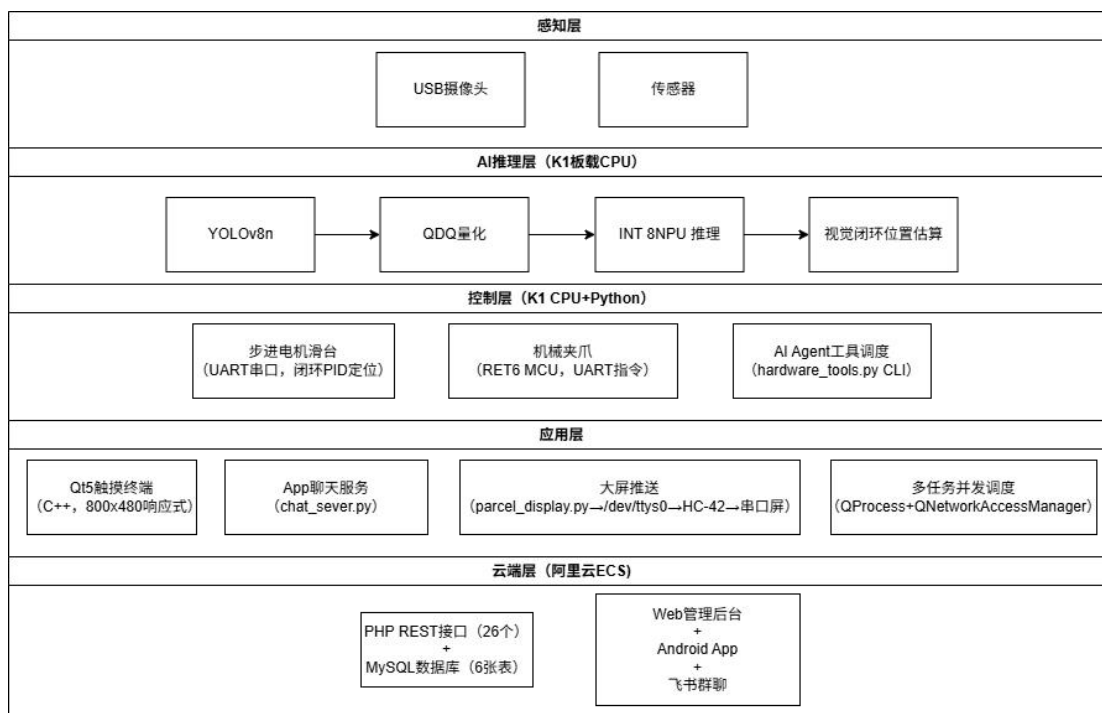


图 2-1 系统整体架构框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件系统由主控计算板、下位机控制板与供电模块、执行机构、交互与显示设备四个子系统组成，各子系统在物理上通过串口、USB、HDMI 等标准接口互联，在逻辑上形成“上层调度、下层执行”的两级控制结构。

(1) 主控计算板——SpacemiT K1 Muse Pi Pro

系统的核心是一块搭载 SpacemiT K1 芯片的 RISC-V 单板计算机，运行 Bianbu Linux 操作系统。K1 在本项目中承担三类任务：一是 AI 推理，板载 NPU 支持 INT8 量化推理，专用于运行 YOLOv8n 目标检测模型，与 CPU 并行工作互不占用；二是图像加速，处理器支持 RVV 1.0 向量指令集，通过 SpacemiT RVV HAL 对 OpenCV 的 cv2.remap 畸变校正核函数进行向量化加速，实测加速比 14.4 倍；三是业务调度，K1 的 CPU 运行全部 Python 控制脚本与 Qt 5 应用程序，统一调度摄像头扫码、视觉追踪、硬件串口通信、云端数据同步、多端服务进程等并发任务。

K1 对外提供多路接口。两路 USB 串口（`/dev/ttyUSB0` 与 `/dev/ttyACM0`）

分别连接步进电机控制器和自研下位机 PCB，承担硬件控制指令的下发；一路板载串口（`/dev/ttyS0`）连接淘晶驰串口屏的 HC-42 蓝牙模块，定时推送包裹到站信息；USB 端口连接摄像头，提供图像采集输入；HDMI 接口连接 IPS 触摸显示屏，承载 Qt 5 用户界面的显示与触摸交互；网络接口接入局域网，支持与云端 MySQL 的数据同步以及 App AI 助手的局域网直连服务。K1 同时运行的七条通信链路全部从这一单节点出发，其异构 I/O 整合能力是本系统硬件架构的核心支撑。

选型依据在于三点的协同：NPU 提供独立 2 Tops AI 算力，视觉推理任务不与 CPU 争抢资源；RVV 指令集对图像处理核函数提供硬件级加速，确保预处理不成为视觉管线瓶颈；RISC-V 架构配套的完整 Linux 生态使得 Python、Qt、OpenCV 等工具链可直接移植，大幅降低软件开发门槛。

（2）下位机控制板与供电模块

机械夹爪的多关节舵机控制需要大量 PWM 通道与精确的时序控制，K1 自身的 GPIO 资源无法满足需求，因此团队自主设计了一块以 STM32F411RET6 为核心的下位机控制板。F411 运行于 100 MHz 主频，具备充足的定时器与 PWM 通道数量，可实现多路舵机的并发控制与精确时序管理。

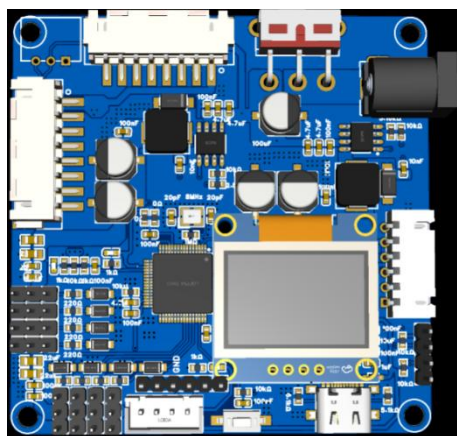


图 2-2 下位机 PCB 仿真图（正）

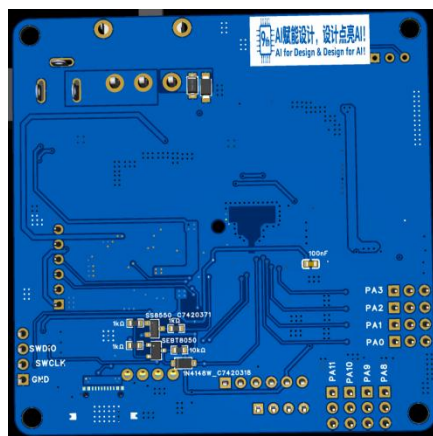


图 2-3 下位机 PCB 仿真图（反）

下位机控制板通过 USB 虚拟串口（`/dev/ttyACM0`，115200bps）接收来自 K1 的字符串指令，解析后输出对应的 PWM 信号，驱动机械夹爪各关节舵机执行抓取、放料、复位等动作。这种两级控制架构将高层业务逻辑与底层实时 PWM 控制分离——K1 发出语义指令（如“CATCHIN”、“DROPOUT”），

RET6 负责将其翻译为具体的舵机时序，两者职责明确，互不耦合。

K1 的供电模块同样为团队自主设计的独立 PCB，非成品模块的堆叠组合，根据 K1 的实际功耗需求定制了稳压与滤波电路，保证 K1 在多任务并发（NPU 推理+串口通信+网络请求同时运行）时供电稳定。

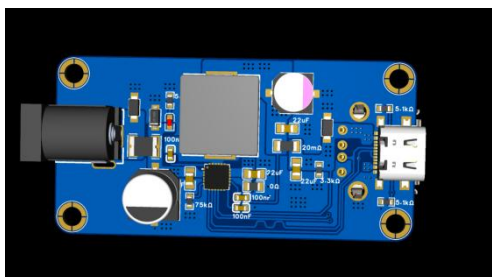


图 2-4 供电模块 PCB 仿真图（正）

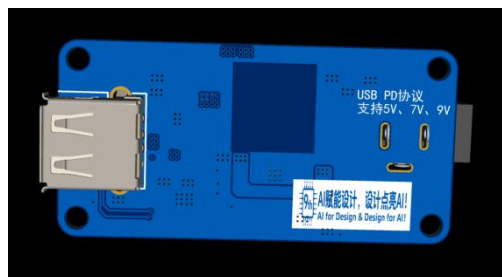


图 2-5 供电模块 PCB 仿真图（反）

（3）执行机构

执行机构是系统与物理包裹直接接触的部分，由步进电机滑台、机械夹爪、传送带三个组件协同构成完整的入库与出库动作链路。

步进电机滑台负责包裹的横向位移，通过 `/dev/ttyUSB0` 与步进电机控制器以自定义二进制帧协议通信。滑台上设有两个独立的步进电机，地址分别为 `0x01` 与 `0x02`，对应奇数格口（1/3/5/7）与偶数格口（2/4/6/8）两侧的出口区域，两路电机可并发运动，支持同时处理两个取件请求，提升站点吞吐效率。电机采用绝对位置定位模式，每次运动前发送清零帧使编码器归零，再发送绝对位置运动帧，四个预设脉冲位置分别对应初始位、格口 1—2 区域、格口 3—4 区域、格口 5—6 区域，重复定位精度由编码器闭环保证。

机械夹爪安装于滑台上方，由 RET6 下位机驱动，负责包裹的抓取与放置动作。入库时，机械夹爪在视觉闭环引导下对准传送带上的包裹，执行 CATCHIN 抓取；出库时，先移至目标格口执行 CATCHOUT 抓取，再随滑台移至出口区执行 DROPOUT 放料。夹爪的每一个标准动作均包含固定等待时间（3 秒），确保多关节舵机完成完整运动行程后再进入下一步，避免因时序提前导致夹持失败。

传送带位于入库通道，负责将到站包裹定向输送至机械夹爪的抓取区域。摄像头置于传送带两侧，当包裹到位时会被视觉识别，从而触发 PID 闭环收敛夹取入库流程。这一触发机制确保了入库流程的全自动驱动，无需人工干

预或定时轮询。

三个执行机构在时序上严格配合：传送带将包裹送到位触发视觉信号，视觉追踪引导步进电机对准包裹，机械夹爪抓取后步进电机将包裹送入格口，整个入库过程环环相扣，任意环节异常均由软件层检测并自动上报工单。

（4）交互与显示设备

系统配备三类显示与交互设备，面向不同的使用场景。

IPS HDMI 可触摸显示屏分辨率为 800×480 ，通过 HDMI 连接 K1，安装于系统前端面向用户。Qt 5 应用程序运行于 K1，将界面输出至该屏幕，用户通过触摸操作完成手机尾号输入、取件确认、信息查询等交互。Qt 界面采用全局响应式样式表，启动时以 800×480 为基准计算全局缩放系数，适配不同分辨率屏幕。

淘晶驰串口屏是站点信息大屏，部署于用户进入站点时视线所及的位置，实时展示当前在库待取包裹的信息，起到“包裹到站广播”的作用。K1 通过 `/dev/ttyS0` 板载串口发送符合淘晶驰协议的指令帧，帧内携带待取包裹的格口编号与收件人手机尾号等信息。由于串口屏的安装位置与 K1 之间不方便走线，系统引入了 HC-42 蓝牙串口透传模块作为无线桥接器：HC-42 的一端通过串口连接 K1 的 `/dev/ttyS0`，另一端通过蓝牙与串口屏配对，在软件层完全透明地转发串口数据，115200 波特率、3 秒推送周期，使串口屏的部署位置不受布线限制。

USB 摄像头连接于 K1 的 USB 端口，是系统获取视觉信息的唯一入口，同时服务于扫码识别与目标检测两条管线。两条管线并非同时占用摄像头，而是由入库流程的不同阶段分时调用：`scan_json.py` 在包裹到达传送带之前完成扫码，获取包裹身份信息；`pack_in.py` 触发后调用视觉追踪管线，接管摄像头完成闭环定位引导；抓取完成后视觉追踪停止，摄像头回归空闲等待下一个扫码周期。

（5）硬件子系统间的连接关系

从信号流向看，整个硬件系统形成一条从感知到执行的完整链路：摄像头将物理场景转化为图像数据，K1 在 NPU 与 CPU 上完成推理与控制计算，通过 USB 串口向步进电机控制器和机械臂下位机发出动作指令，执行机构

驱动包裹完成物理位移，传感器反馈信号关闭闭环；与此同时，K1 通过串口屏协议向大屏推送状态信息，通过 HDMI 向触摸屏输出用户交互界面，通过网络与云端保持数据同步。K1 是这条链路上唯一的汇聚节点，所有感知输入在此汇合，所有控制输出从此发出，既是计算中枢，也是通信枢纽。

2.2.2 机械设计介绍

本机械结构采用模块化分层设计理念，以工业铝型材搭建主体承载框架，集成物料输送、分拣执行、视觉采集、人机交互四大功能单元，整体布局规整，各组件安装定位精准，可完整实现物料上料、传输、识别、分拣归类的全流程作业，满足智能分拣场景的功能与精度需求

（1）整体机架结构

系统主体采用工业铝型材拼接成立体机架，通过专用连接件刚性固定，结构强度高、装配拓展灵活，可稳定承载各功能模组的动静载荷。机架内侧配置木质安装基板，为各组件提供标准化安装平面；机架设置线缆固定卡扣，动力与信号线路沿型材规范布设，整体布线整洁，便于后期维护与功能升级。

（2）皮带输送机构

输送单元以平皮带传送带为物料流转核心，由直流减速电机驱动，可配合控制系统调节输送速度，适配不同分拣节拍。传送带两侧设置物料暂存工位与分类料区，配合执行机构完成物料取放与分类收纳，形成闭环物料流转路径。

（3）直线分拣执行机构

传送带两侧对称布置两套步进驱动直线滑台模组，构成分拣执行单元，可实现水平方向高精度位移，配合执行末端完成物料的推送分拣。模组支架预留多组调节孔位，可灵活调整安装高度与位置，适配不同尺寸物料的分拣作业。两套模组协同工作，可根据识别结果将物料精准推送至对应分类区域。

（4）视觉采集与人机交互单元

机架顶部架设竖向支架，固定视觉采集摄像头，镜头正对传送带分拣工位，实时采集物料图像，为后端识别算法提供数据输入。机架正面安装触控显示屏，作为人机交互界面，可实时显示系统运行状态、分拣数据，支持参

数设置与启停操控，实现可视化运维。

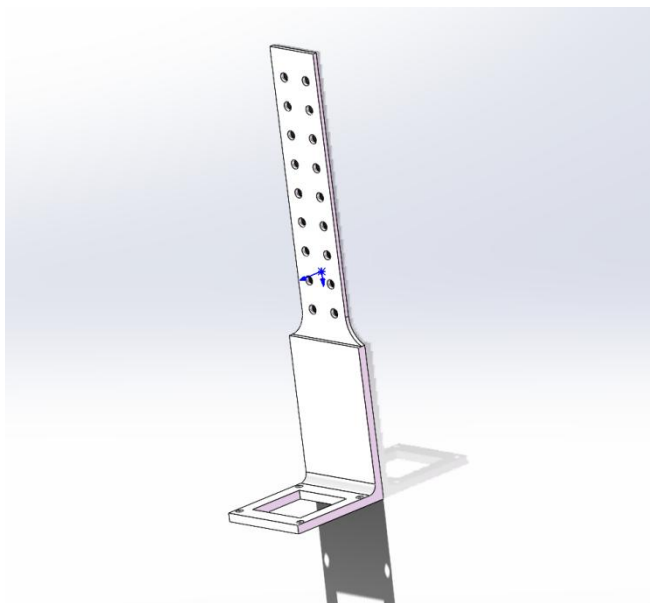


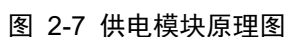
图 2-6 摄像头支架模型

2.2.3 电路各模块介绍

本电路为嵌入式竞赛设计的控制板，以 STM32F411RET6 微控制器为核心，整体分为电源模块、主控模块、外设接口与驱动模块三大部分，实现电源转换、运算控制、电机驱动、多外设扩展等功能，以下分模块介绍。

（1）电源模块

电源模块为整板提供稳定、分级的直流供电，是系统可靠运行的能量基础。模块核心采用 TPS5450 同步降压 DC-DC 转换芯片，该芯片支持宽范围电压输入，最大持续输出电流可达 5A，具备转换效率高、带载能力强的特点，配合外围电感、续流二极管、反馈分压网络与自举电容构成完整的降压拓扑结构。系统供电由 DC1 接口接入 12V 直流电源，经船型总开关 SW1 实现整机电源通断控制，输入侧采用电解电容与陶瓷电容组合的滤波网络抑制电源纹波与输入噪声。经两级 DC-DC 降压转换后，模块最终输出 6V、5V、3.3V 三路独立电源轨：6V 电源为舵机等外设供电，5V 电源为驱动芯片与接口电路供电，3.3V 电源为主控芯片与数字逻辑电路供电。各路电源输出端均配置大容量滤波电容以保障电压稳定性，同时设置电源状态指示灯，可直观判断各级电源的工作状态，满足传送带电机启停、换向等动态负载下的



(2) MCU 主控

MCU 主控模块是整个传送带控制系统的运算与控制核心，承担运行逻辑调度、电机调速控制、外设数据交互与状态监测等核心功能。模块主控芯片采用 STM32F411RET6 32 位微控制器，该芯片基于 ARM Cortex-M4 内核，集成 DSP 与浮点运算单元，最高主频可达 100MHz，配备丰富的 GPIO、定时器、串口、ADC 等外设资源，可充分满足传送带启停控制、速度调节、多外设联动等场景的控制需求。

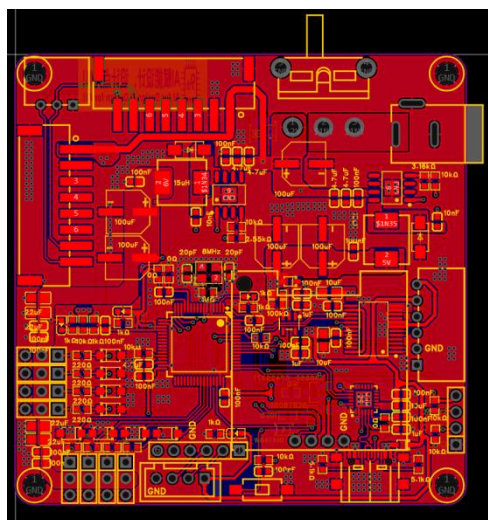


图 2-8 下位机 PCB 布线图 (正)

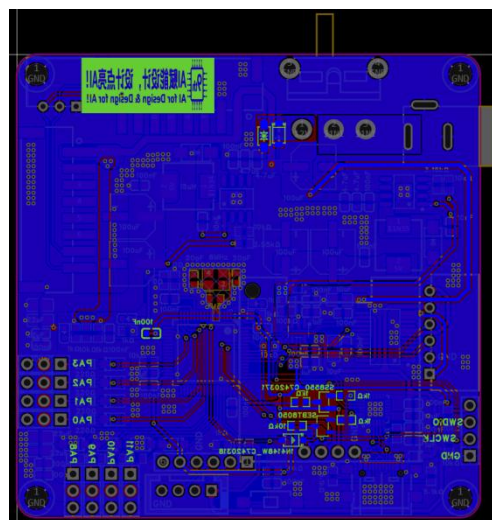


图 2-9 下位机 PCB 布线图 (反)

(3) 外设接口与驱动模块

外设接口与驱动模块是主控单元与外部执行、交互设备的连接枢纽，包含传送带电机驱动电路、上位机通信电路与多类外设扩展接口，所有信号链路均设计静电防护电路，提升系统现场运行的可靠性。

传送带电机驱动电路以 TB6612FNG 双路 H 桥电机驱动芯片为核心，该芯片内置功率驱动拓扑，持续输出电流 1.2A，峰值电流可达 2A，可匹配传送带直流驱动电机的功率需求。芯片功率侧接入 12V 动力电源，为电机提供动力输入；逻辑侧采用 5V 供电，与主控 IO 电平兼容。芯片的方向控制引脚 AIN1/AIN2、BIN1/BIN2 与调速引脚 PWMA、PWMB 均连接至 STM32 主控的 GPIO 与定时器引脚，主控通过输出高低电平控制电机的启停与正反转，实现传送带的正向输送、反向回退与停机操作；通过输出不同占空比的 PWM 信号调节电机转速，以适配不同的物料输送速度要求。驱动输出端通过 XH2.54 规格接口座连接传送带电机，芯片周边配置去耦电容，抑制电机启停、换向过程中产生的电源尖峰干扰，保障驱动电路运行稳定。

上位机通信电路以 CH9102F USB 转串口芯片为核心，实现 USB 总线与 TTL 串口的电平转换，搭建主控芯片与上位机之间的全双工串行通信链路，是系统与上位机交互的核心通道。通过该通信通道，上位机可向主控系统下发传送带运行参数、控制指令与配置信息，主控端可向上位机回传实时运行状态、物料计数数据与故障告警信息，为系统调试、运行监控与参数整定提供稳定的数据交互支撑。同时该芯片配合一键下载电路，可自动控制主控芯片的复位信号与启动电平，无需手动切换 BOOT 模式即可完成固件程序烧录，兼顾程序下载功能，提升开发调试效率。

外设扩展接口区域集成多类标准化接口，所有接口信号路径均串联限流电阻、并联 SMF5.0A 瞬态抑制二极管实现 ESD 静电防护，接口统一采用 XH2.54 规格座子，具备良好的连接可靠性与扩展性。舵机接口接入 6V 电源与主控定时器引脚，通过 PWM 信号控制舵机角度，可配合传送带完成物料分拣。

2.3 软件系统介绍

2.3.1 软件整体介绍

本系统软件由五个子系统构成，运行于两个物理域：K1 本地边缘节点与阿里云 ECS 云端服务器。K1 本地运行 Qt 5 触摸终端与全部 Python 功能脚本，承担视觉感知、运动控制、AI Agent 调度、多端联动轮询等实时任务；云端运行 PHP REST 接口层、MySQL 数据库与 Web 管理前端，承担数据持久化、多端权限管理与 Web 交互；React Native App 运行于用户的 Android 手机，通过 HTTPS 与云端通信，在局域网内通过 HTTP 直连 K1；飞书群聊通过 OpenClaw 飞书渠道接入，无需专用客户端。五个子系统共用同一套云端数据层，包裹状态的权威来源唯一，保证多端强一致性。

子系统	运行位置	主要职责	对外接口
Qt 5 触摸终端	K1 本地	用户本地交互、子进程统一调度、AI 助手入口	HDMI 触摸屏、QProcess、QNetworkAccessManager
Python 脚本层	K1 本地	视觉感知、硬件控制、AI 工具执行、App 轮询	串口设备、摄像头、MySQL、HTTP
PHP 云端接口	阿里云 ECS	26 个 REST 接口，统一为所有端提供数据服务	HTTPS（所有端）
Web 管理后台	浏览器	管理员数据管理与分析，纯原生前端	HTTPS（PHP 接口）
React Native App	Android	远程取件、移动管理、AI 助手	HTTPS（云端）+ HTTP（K1 局域网）

（1）K1 本地软件结构

K1 本地的软件以 Qt 5 应用程序为顶层调度中枢，负责界面渲染、用户事件响应与子进程生命周期管理。Qt 进程不直接操作串口硬件，而是通过 QProcess::startDetached 异步启动独立 Python 子进程执行硬件动作，通过 QNetworkAccessManager 的异步接口访问云端 REST 接口，UI 主线程全程不阻塞。

Python 脚本层分为两类：一类是由 Qt SettingsDialog 管理的长生命周期常驻进程，包括 scan_json.py（摄像头扫码，持续运行）、parcel_display.py（大屏推送，持续运行）、pack_in.py（入库驱动，持续等待传感器信号）、chat_server.py（App AI 聊天服务，由 systemd 用户服务管理，开机自动启动）；另一类是短生命周期的按需触发脚本，包括 motor.py（单次出库，执行完成后自动退出）、pickup_poller.py（每 3 秒轮询一次，Qt 启动时自动后台运行）。

两类脚本通过数据库进行状态共享，而非直接通信：scan_json.py 写入

包裹入库记录，pack_in.py 读取格口分配；pickup_poller.py 读取 pending 取件请求，motor.py 执行出库后回写 done 状态；hardware_tools.py 读写异常工单表。这种“数据库作为消息总线”的设计使各脚本高度解耦，单个脚本崩溃重启不影响其他脚本的运行状态。

(2) 云端软件结构

云端 PHP 接口层按职责分为六组：用户认证（登录/注册/验证码/改密/重置密码）、包裹管理（CRUD/CSV 批量导入导出/扫码入库/游客查件）、取件联动（请求写入/状态轮询/完成回写/撤销纠错）、异常处理（工单查/编/删/筛选导出）、推送通知（Expo Token 注册）、设备注册（K1 局域网 IP 动态注册与查询）。共 26 个接口，统一由 auth.php 中间件处理鉴权，支持 Web PHP Session 与 App/Qt Bearer Token 双模式，同一套接口无需维护双份实现。

数据库名称: qiansai 大小: 160.00KB

☐	表名	备注	引擎	字符集	行数	大小	操作
☐	anomaly_report	异常信息	InnoDB	utf8mb4_general	1	16.00KB	导出表结构 修复 优化 转换引擎
☐	pickup_request	App的取件请求	InnoDB	utf8mb4_general	14	16.00KB	导出表结构 修复 优化 转换引擎
☐	push_tokens	Expo推送Token	InnoDB	utf8mb4_general	0	16.00KB	导出表结构 修复 优化 转换引擎
☐	shipments	快递信息	InnoDB	utf8mb4_general	2	16.00KB	导出表结构 修复 优化 转换引擎
☐	user_tokens	App登录Token	InnoDB	utf8mb4_general	60	16.00KB	导出表结构 修复 优化 转换引擎
☐	users	用户账号	InnoDB	utf8mb4_general	8	16.00KB	导出表结构 修复 优化 转换引擎

图 2-10 MySQL 表单

网站名	状态	备份	根目录	日志量	到期时间	备注	PHP	SSL证书	操作
red-waltz.top	运行中	有(1)	/www/wwwroot/red-waltz.top	4.64 MB	永久	red-waltz.top	8.1	剩余89天	统计 WAF 设置 删除

☐ 请选择批量操作

1/10页/页 共 1 条 前往 1 页

图 2-11 域名管理

Web 管理后台为纯原生前端，以 HTML5/CSS3/JavaScript ES6+构建，所有数据可视化均由纯 CSS Flex/Grid 与 JS 动态宽度实现。页面共用 theme.js 与 theme.css 实现深色/浅色主题切换，偏好存入 localStorage，刷新页面或跨标签页保持一致，切换时 CSS transition 保证 0.25s 平滑过渡。

(3) App 软件结构

React Native App 基于 Expo SDK 56 构建，通过 EAS Build 云端构建 Android APK，无需本地 SDK 或 Xcode 环境。采用 React Navigation 7 的 Stack+BottomTabs 双层导航，角色路由在 App 启动时根据 expo-secure-store 中存储的 Bearer Token 与用户角色决定进入用户标签页或管理员标签页。全部动画基于 React Native Animated API 实现，严格区分 native driver 与 JS driver 使用场景，保证渲染性能。runtimeVersion=appVersion 策略支持 OTA 热更新，JS 层变更无需重新打包，通过 eas update --channel production 推送后，已安装的 App 下次启动自动更新。

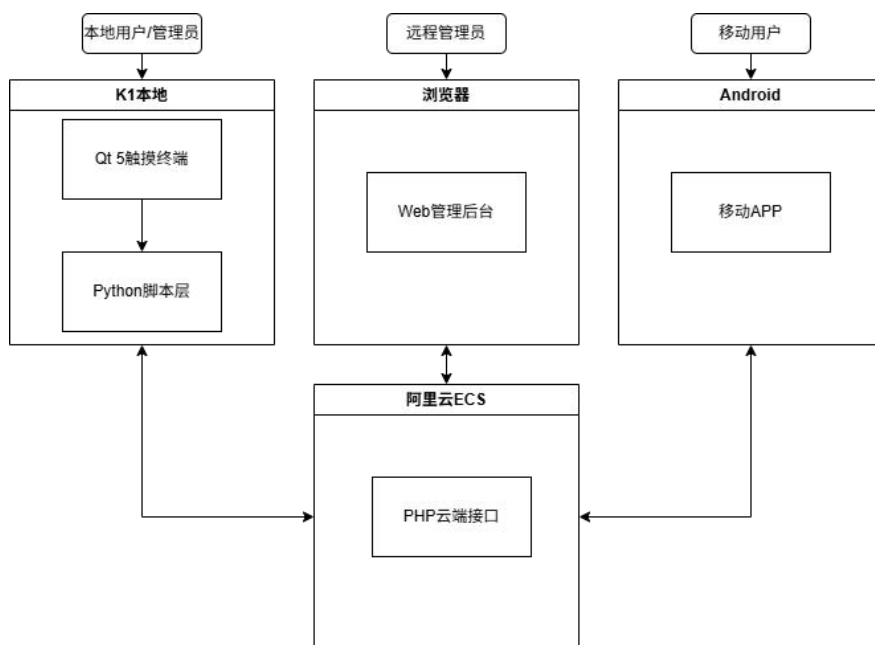


图 2-12 软件结构图

2.3.2 软件各模块介绍

(1) 扫码识别管线

扫码识别管线由 scan_json.py 实现，负责在包裹到达传送带前完成身份信息采集与格口分配，其输出是后续入库流程的唯一数据来源。

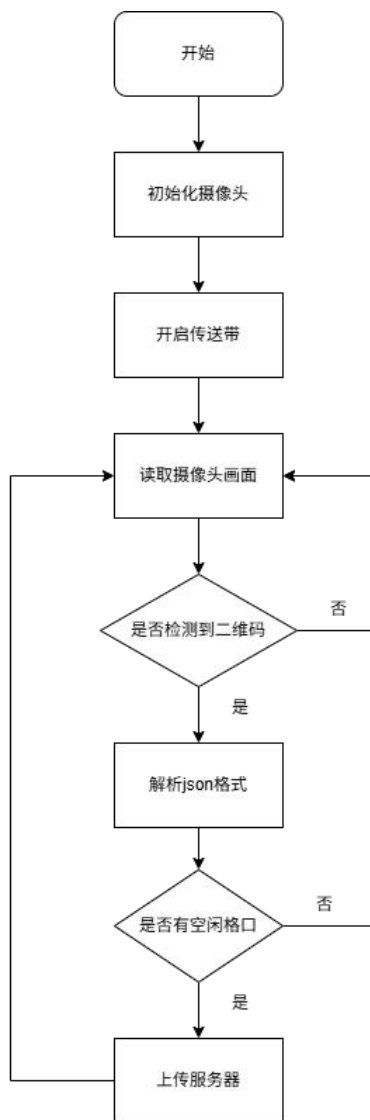


图 2-13 扫码登记流程图

流程中未涉及的还有畸变校正步骤（`cv2.remap`）借助 SpacemiT RVV HAL 向量加速，映射查找表在首次调用时预计算缓存，后续每帧仅执行查表运算，单帧耗时 3.36ms，是纯 Python 实现的 1/14.4，确保校正不成为扫码管线的性能瓶颈。格口分配在 Python 端随机选取后，由 `receive_shipment.php` 在 INSERT 前再次 SELECT 确认格口仍为空闲，若冲突返回 409，脚本重新选取新格口，构成三层格口冲突防护的完整闭环。该管线的关键输入为摄像头原始帧，关键输出为落库的包裹记录（含快递单号、手机号、物品分类、格口编号），该记录随后由 `pack_in.py` 读取，驱动机械臂完成物理入库动作。

（2）运动控制模块

运动控制模块是系统与物理包裹直接交互的执行层，入库由摄像头视觉

信号触发，路径为传送带→格口；出库由上层应用触发，路径为格口→出口区，两条流程共用底层串口控制函数但驱动来源与动作序列不同。

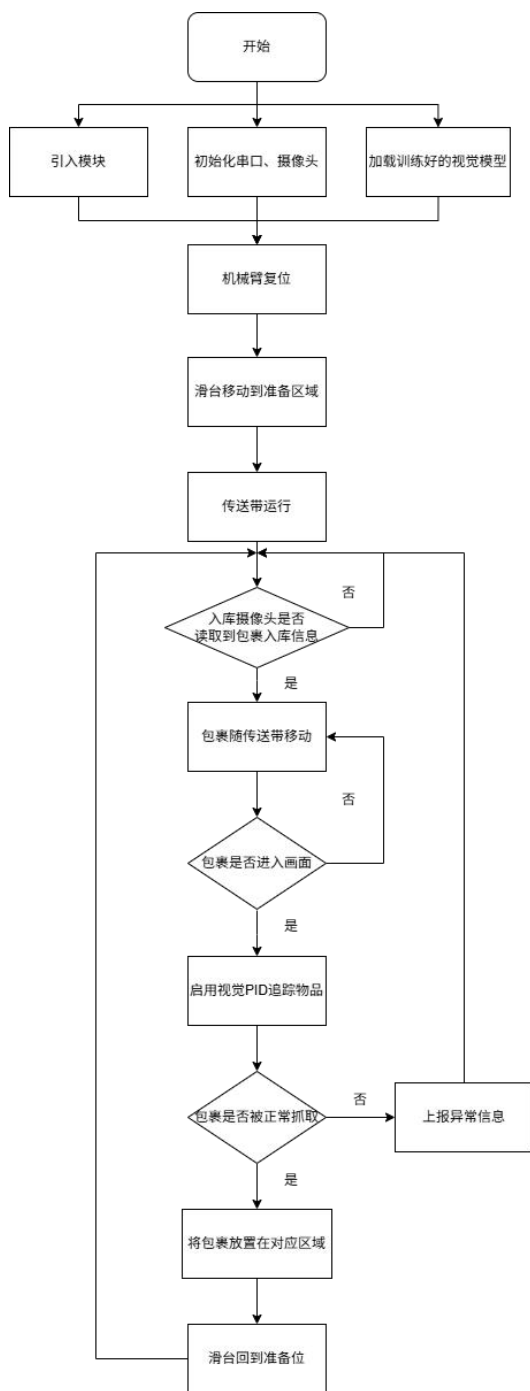


图 2-14 系统入库流程图

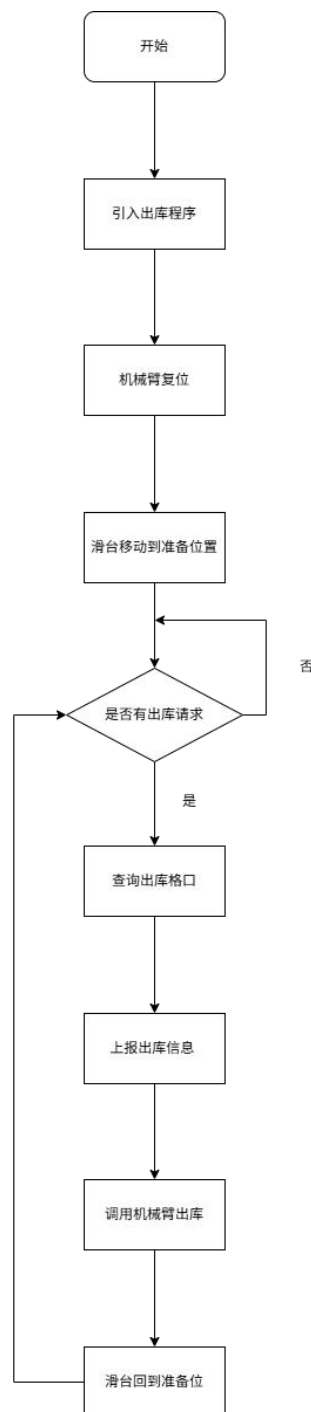


图 2-15 系统出库流程图

步进电机每次运动前先发送清零帧使编码器归零，再发送绝对位置运动帧，保证重复定位精度不受累积误差影响。机械臂每条指令发送后强制延时100ms 并调用 flush()，该延时并非保守估算，而是针对 K1 多任务并发时串口写入时序干扰导致 RET6 MCU 漏指令问题的定向解决方案——该问题仅

在多任务集成运行时偶发，单独测试串口时无法复现，定位耗时较长。出库路径中奇数格口(1/3/5/7)送至左出口区(电机地址 0x01)，偶数格口(2/4/6/8)送至右出口区(电机地址 0x02)，两路电机地址独立，可同时接收两条出库指令并发执行，互不干扰，提升站点吞吐效率。motor.py 始终通过 QProcess::startDetached 或 subprocess 异步启动，调用方不等待进程结束，Qt 界面与出库动作完全解耦。

(3) 自然语言到硬件动作的转化路径

AI Agent 层将飞书、App、Qt 终端的自然语言指令统一转化为硬件动作，三端共用同一套工具实现，调用路径存在差异，但最终均收敛至 hardware_tools.py。

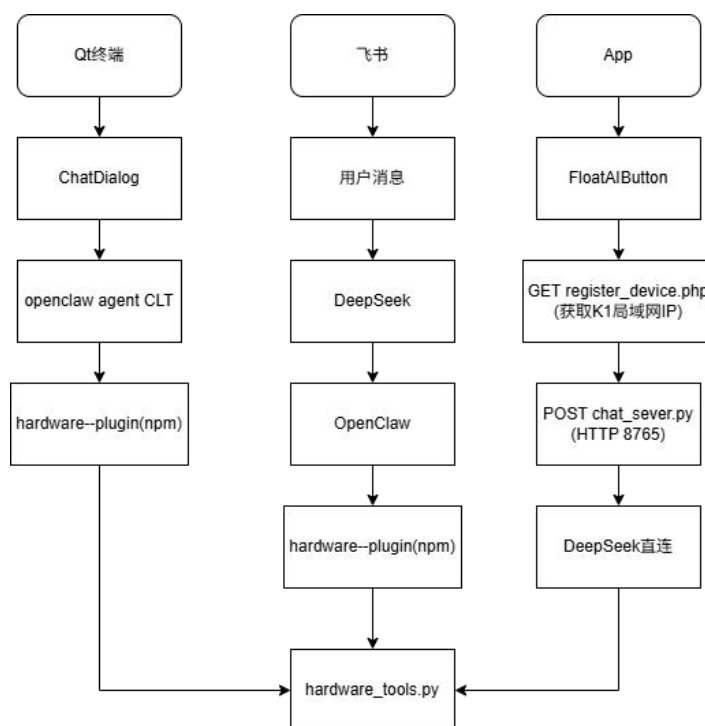


图 2-16 AI Agent 三端调用链路图

三端路径的核心差异在于 App 绕过了 OpenClaw Node.js 进程，直接向 K1 本地 chat_server.py 发起请求，后者直连 DeepSeek API，响应时间大幅缩短。K1 局域网 IP 由 chat_server.py 启动时以及每 5 分钟自动向云端 register_device.php 续报，TTL 10 分钟，App 发起对话前查询该接口动态获取地址，无需手动配置，适应热点切换场景。AI 与确定性代码之间的边界通过 JSON 协议划定：LLM 只负责意图理解与参数提取，输出严格约束为含

message 与 actions 数组的 JSON 结构,硬件执行路径完全由确定性代码驱动,兼顾智能性与可靠性。预约取件工具 hw_prepare_pickup 通过 spawn+detach 启动独立倒计时进程,用户收到 AI 确认回复即可结束对话,到时自动出库并推送大屏通知,对话响应与硬件动作完全异步解耦。

(4) 六表结构与包裹全生命周期数据设计

云端 MySQL 数据库由 6 张表构成,承载包裹全生命周期的持久化存储,是系统唯一的数据权威,所有端的业务状态均以此为准。

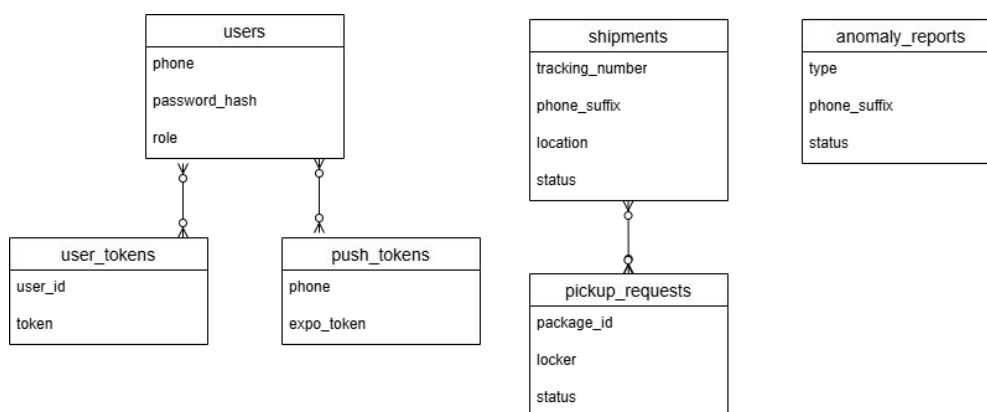


图 2-17 数据库 ER 图

pickup_requests 表的设计意图是充当 App 与 K1 硬件之间的异步消息队列,而非传统意义上的业务数据表——App 写入 pending 请求, K1 消费并回写 done/failed, 两端不直连, 通过表中的 status 字段交换状态, 断网恢复后轮询自动续处理。这一设计使取件链路在网络抖动、K1 重启等异常场景下天然具备恢复能力。anomaly_reports 表通过 type 字段区分入库异常与出库异常, 两类异常来源不同但统一存储, 管理员可在 Web 与 App 端按类型筛选处置, 形成可追溯的完整异常闭环。鉴权体系中 user_tokens 表为每次登录生成独立 token, auth.php 中间件同时支持 Bearer Token 与 PHP Session 两种校验路径, 同一套接口无需为 Web 端与 App 端维护双份实现。

第三部分 完成情况及性能参数

3.1 整体介绍

系统整体为一体化框架结构，各硬件模块按功能分区布置于框架不同位置，协同完成包裹的入库、存储与出库全流程。

系统正面面向用户，最前方固定有触摸显示屏，用户在此完成取件操作与信息查询等全部本地交互。屏幕下方设有传送带，负责将到站包裹向内输送至抓取区域。传送带前端安装有摄像头，负责扫描包裹条码，完成单号识别与入库登记。

框架左右两侧各设有一条滑台，滑台上固定有机械臂，负责抓取传送带上的包裹并将其存入对应格口，或从格口取出包裹送至出库区。框架上方同样安装有摄像头，专门用于辅助入库时的包裹定位，引导机械臂准确对准抓取目标。

框架内部左右两侧设有格口，用于存放在库包裹；格口两端各设有出库区，机械臂完成取件后将包裹放置于此，用户到场领取。

系统背面固定有串口屏，实时显示当前在库包裹信息与取件提醒，方便用户在进入站点时第一时间了解自己的包裹状态。背面同时固定下位机控制板，负责接收上层指令并驱动机械臂各关节动作，是机械执行部分的控制核心。

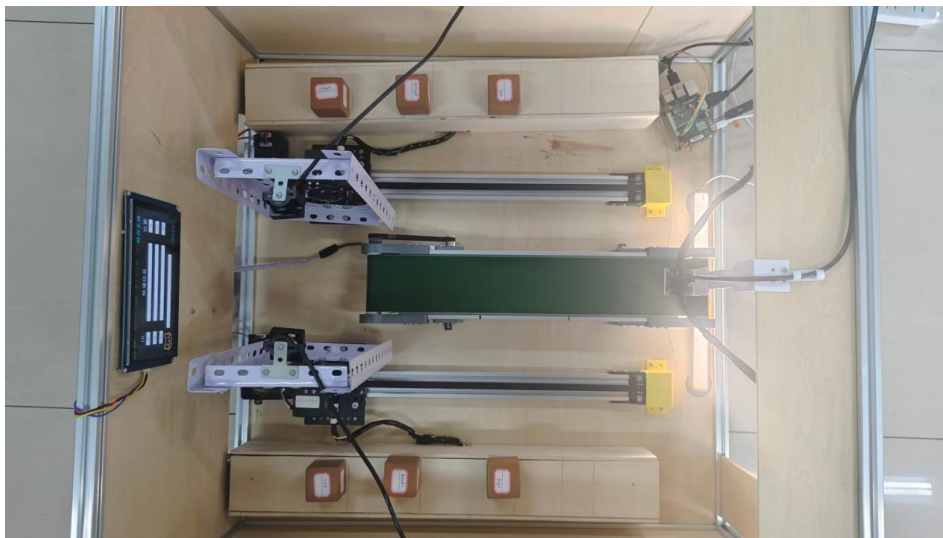


图 3-1 系统俯视图

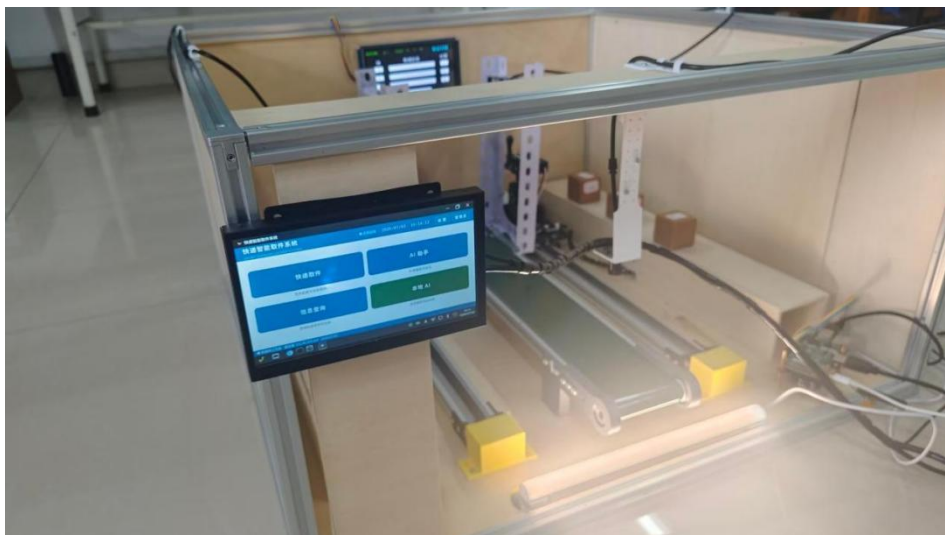


图 3-2 系统 45° 斜视图

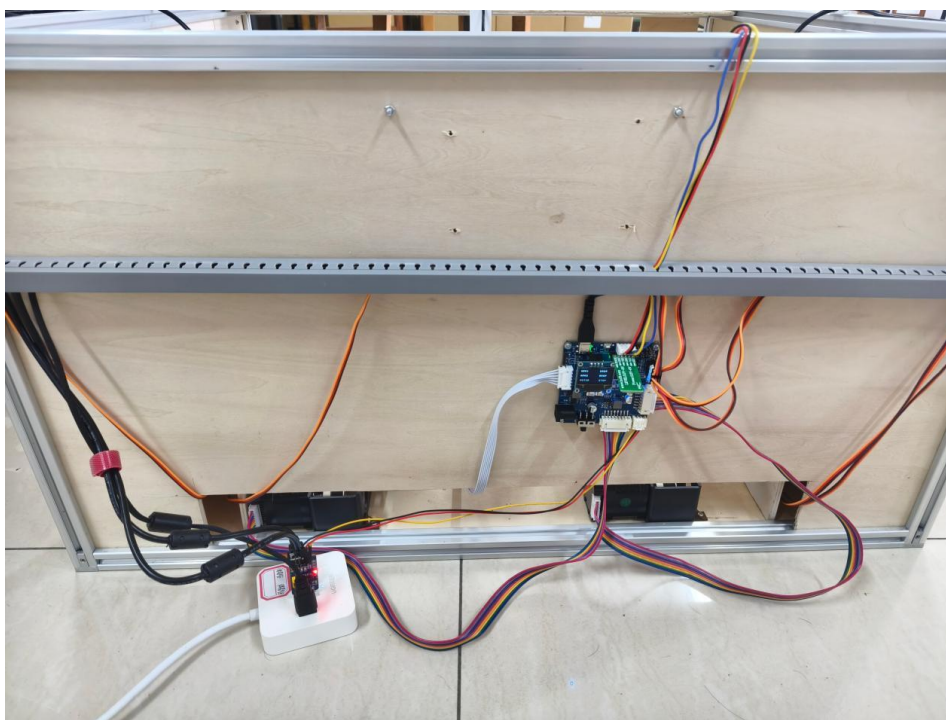


图 3-3 系统背部展示图

3.2 工程成果

3.2.1 机械成果

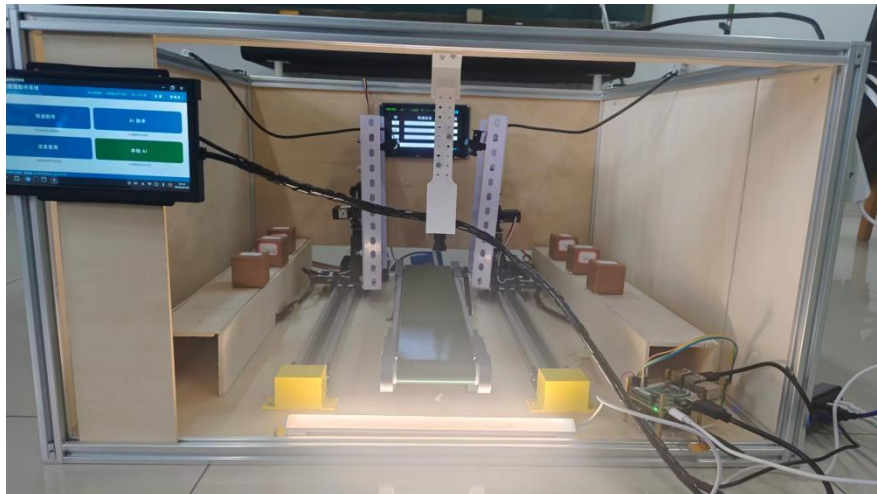


图 3-4 系统整机正面照

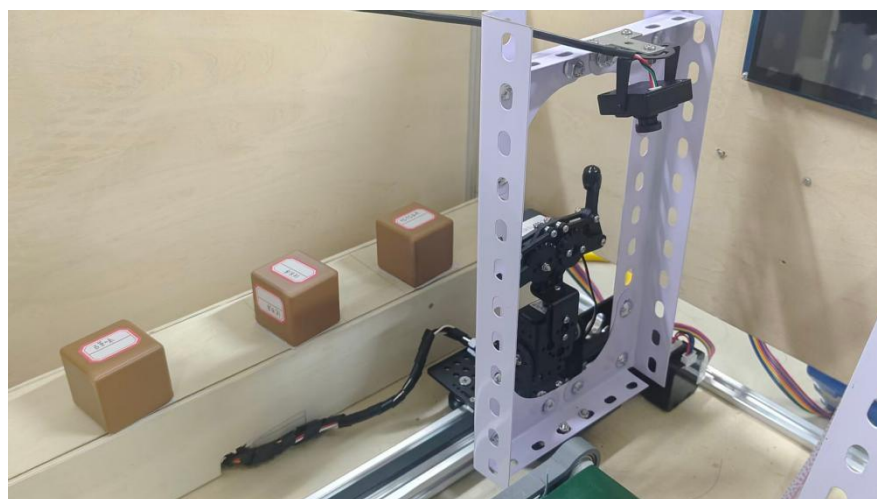


图 3-5 机械臂与步进电机滑台



图 3-6 传送带与登记摄像头

3.2.2 电路成果



图 3-7 K1 开发板俯视图

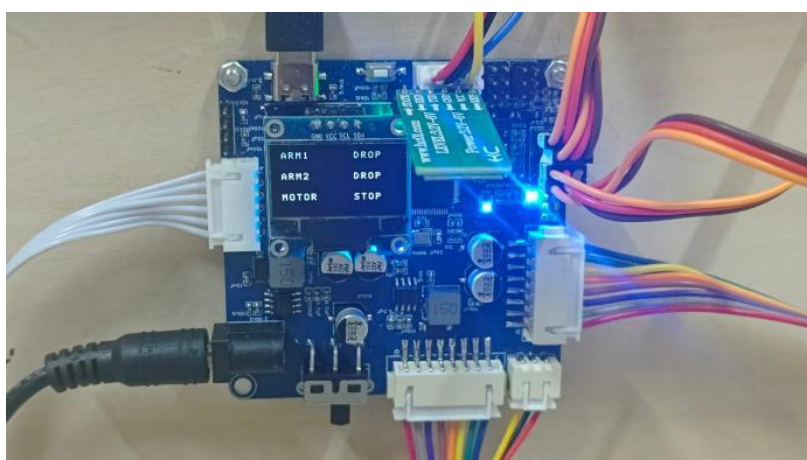


图 3-8 下位机控制板实拍图



图 3-9 K1 供电模块实拍图

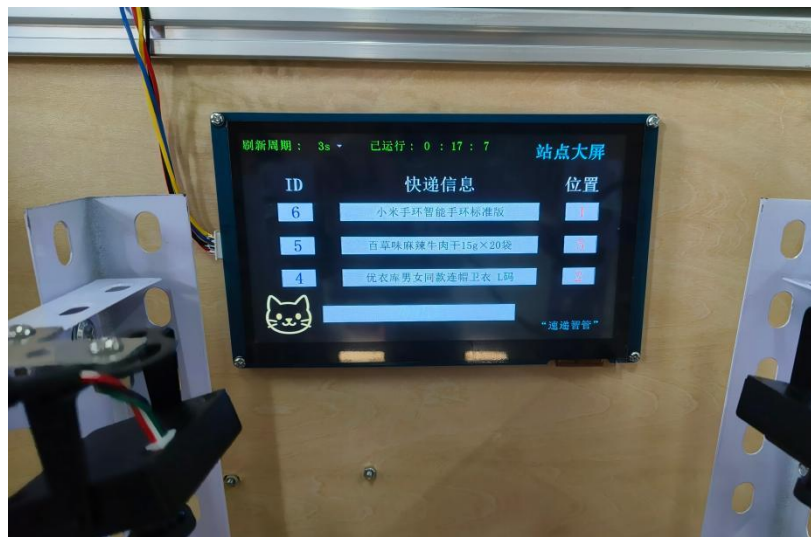


图 3-10 淘晶驰串口屏显示效果图

3.2.3 软件成果

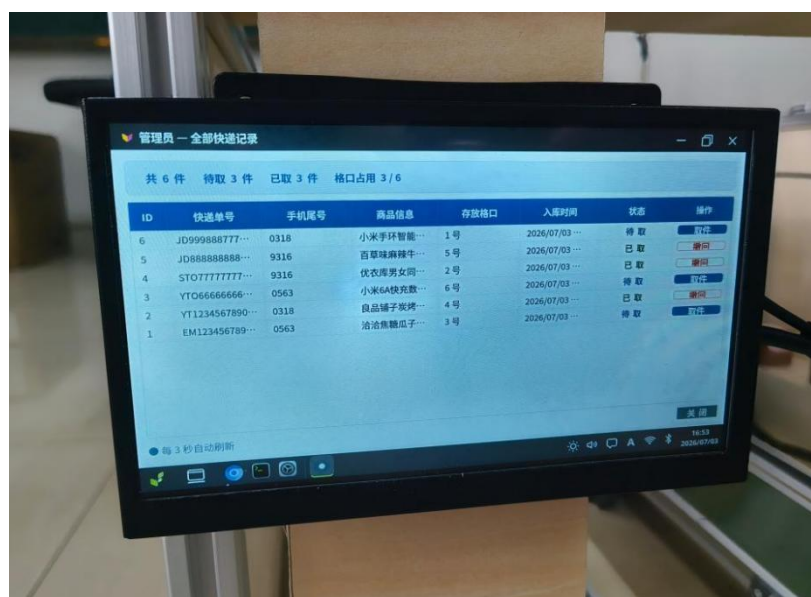


图 3-11 QT 管理员界面图

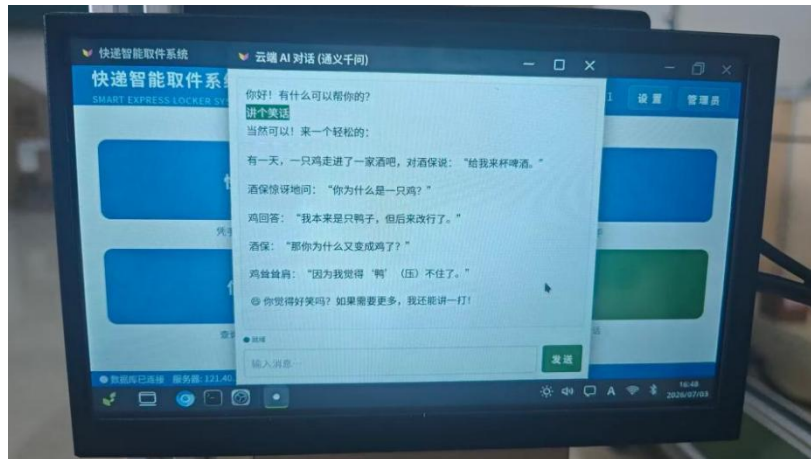


图 3-12 QT AI 对话效果图

5

速速管理

商家信息

包裹列表

数据介绍

入库管理

订单记录

订单处理

管理页

个人中心

退出

Q

搜索单号、手机号、商家...

共 3 条

导出 CSV

删除

ID	快递单号	手机号	商家名称	件数	收货位置	商家地址	入库时间	操作
6	320998887776655	138295488158	小羊子玩具店干马松店	1 件	34401345		2026/7/9 12:54:11	编辑 删除
3	Y706666666666666	13308888543	小羊6A店外数据店2m	6 件	34401345		2026/7/9 12:53:45	编辑 删除
1	895214852780CN	13308888543	品山市数据店25g-10段	3 件	34401345		2026/7/9 12:53:27	编辑 删除

5

10条数据

数据总数: 16/47-58

图 3-13 Web 包裹列表

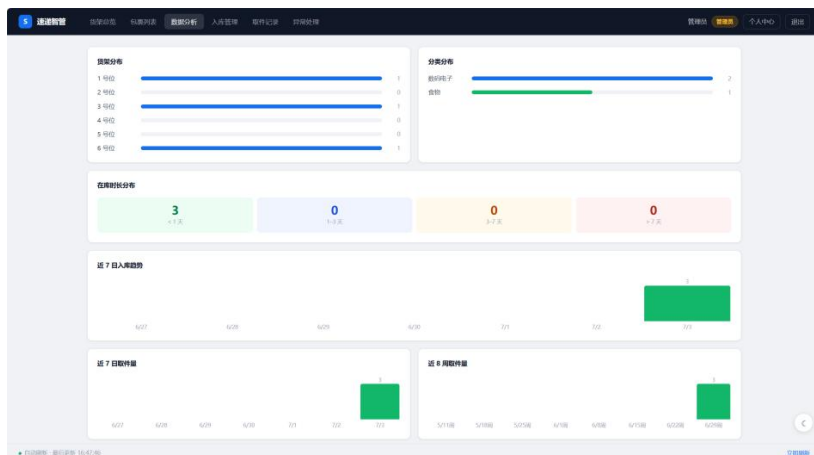


图 3-14 Web 数据看板

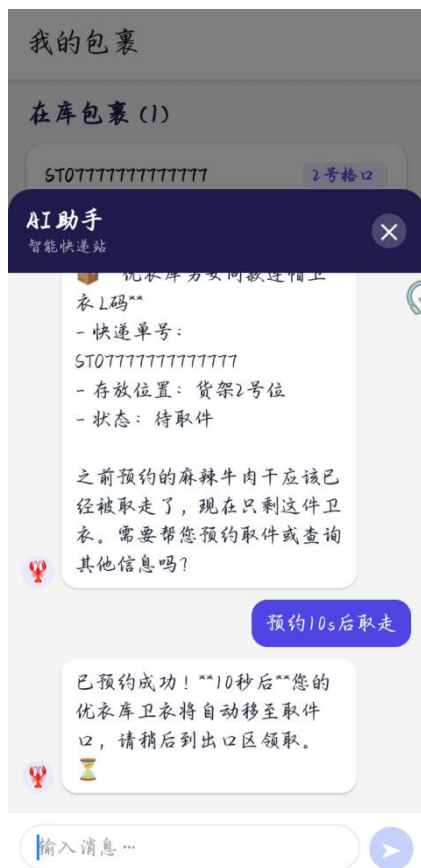


图 3-15 APP 悬浮 AI 助手对话界面



图 3-16 APP 异常工单处理界面

3.3 特性成果

(1) 视觉感知性能

指标	结果
RVV 加速畸变校正	3.36ms/帧，较纯 Python 提速 14.4 倍
YOLOv8n NPU 推理帧率	稳定 20+FPS，视觉管线无瓶颈
目标检测置信度	持续>0.95，测试期间误识别率 0%
视觉闭环对齐精度	均值 1.5mm，最大偏差 3.3mm（50 次实测）
追踪收敛耗时	典型值 274ms（4—6 次 PID 迭代）

(2) AI Agent 取件

自然语言输入到机械臂动作全流程测试稳定，AI 正确解析手机尾号、准确返回格口编号，结构化 JSON 响应解析成功率 100%。App AI 助手直连 DeepSeek 后响应时间较原 OpenClaw 路径缩短约 50%，三端工具调用路径均通过联调验证。

(3) 多端功能覆盖

入口	自助取件	AI 取件/ 预约	游客查件	包裹管理	数据分析	异常处理
Qt 触摸终端	√	√（含预约）	—	√（管理员）	—	—
Web 后台	—	—	√	√	√	√
Android App	√	√（含预约）	√	√（管理员）	√	√（管理员）
飞书群聊	—	√（含预约）	—	—	—	√（上报）

(4) 数据管理

包裹全生命周期（入库→在库→取件→历史）完整记录于 MySQL，支持按手机号/单号/日期多维检索；CSV 批量导入前端实时预览校验，合法行绿色高亮、错误行红色标注；取件记录与异常工单按筛选条件导出 CSV，中文字段自动加 BOM 头解决 Excel 乱码。

(5) 系统集成稳定性

App 一键取件完整链路（App 发起→云端入队→K1 轮询→motor.py 出库→回写状态→App 反馈）在测试中稳定运行；pickup_poller.py 90 秒超时保护机制有效防止电机卡死时永久阻塞后续请求；子进程停止时 SIGINT 归位机制确保硬件在任何中止场景下安全复位。

第四部分 总结

4.1 可扩展之处

①**格口规模扩展**：步进电机的目标位置由脉冲数配置表决定，增加格口仅需扩展配置项与滑台行程，软件层无需改动，可平滑扩展至更大规模的存储阵列。

②**多模态身份核验**：现有取件依赖手机尾号输入，后续可集成 NFC 刷卡或人脸识别模块，进一步降低操作门槛，同时提升安全性、防止冒领。

③**多站点联网管理**：数据库增加站点字段后可扩展为多站点统一管理平台，总部管理员可跨站点查看包裹分布与异常工单，支持区域协调调配。

④**异常自愈机制**：现有异常工单需人工处置，后续可对高频异常（如特定格口重复出库失败）自动下发停用指令并推送维护提醒，实现故障预测与自动隔离。

⑤**离线语言能力**：当前语言推理依赖云端 API，K1 NPU 空闲时段可调度运行量化小参数语言模型，实现断网环境下的基础 AI 对话能力。

4.2 心得体会

本系统是团队从零搭建的完整工程项目，横跨硬件设计、嵌入式开发、云端服务、移动应用四个方向。回顾整个开发过程，收获最深的不是某个单点技术的突破，而是在把所有模块真正联调起来的过程中，对“系统工程”这四个字有了更具体的理解。

硬件从设计到调通比预期复杂。PCB 打板后的第一次上电，供电模块输出电压存在纹波，导致 K1 启动时偶发重启。排查过程中逐一排除软件侧可能，最终定位到滤波电容选型偏小，换用更大容值电容后问题消失。这次经历让我们意识到，硬件问题在系统集成阶段往往以软件症状的形式暴露出来，排查路径必须软硬件并行考虑。机械结构的装配同样花费了大量时间，夹爪安装座的定位孔在第一版加工件上存在微小偏差，导致机械臂在极限行程处与滑台框架发生干涉，最终通过修改安装座设计并重新加工解决。

多协议并发的调试代价超出预估。系统集成后同时运行七条通信链路，每条协议在单独测试时均工作正常，但放在一起运行就会出现难以复现的偶

发问题。其中印象最深的是机械臂漏指令的排查：现象是机械臂偶尔跳过某个动作，但串口抓包看不出异常。经过长时间对比测试，最终发现是 K1 在多任务并发时，MySQL 查询与串口写入在 Python 事件循环中产生了细微时序干扰，导致两条指令之间的间隔偶尔低于 MCU 的处理时间。加入固定延时与 flush() 后问题消失。这次排查让我们形成了一个判断：多协议系统的调试难度不是各协议难度的简单叠加，而是时序交叉产生的组合复杂度，单独测试永远无法覆盖集成后的全部情况。

让数据只有一个家，是多端系统最重要的架构决策。项目中期曾出现过 Qt 本地缓存的包裹状态与云端 MySQL 不同步的问题，导致机械臂已完成出库但界面仍显示待取，用户重复确认了两次。事故之后团队确立了一条原则：所有端的业务状态只在 MySQL 维护，任何端不持有副本。这个原则执行起来需要付出持续轮询的网络开销，但换来了多端数据的强一致性，后期再没有出现过状态分歧的问题。取件消息队列的设计也源于类似的思考——App 与 K1 不直连，通过数据库中的状态字段交换信息，断网重连后流程自然续接，比任何重试逻辑都可靠。

整个项目让我们最真切地感受到：一个系统的复杂度很容易被低估，因为每个部分单独看都不难，难的是它们组合在一起时涌现出来的问题。而应对这种复杂度最有效的方式，是从一开始就把边界划清楚——模块边界清晰，联调时才有地方可查；状态边界清晰，多端协作才不会产生分歧。

第五部分 参考文献

- [1] 进迭时空技术团队. 进迭时空官方技术文档[EB/OL]. [2026-06-16].
<https://www.spacemit.com/community/document>.
- [2] STMicroelectronics. STM32F411RC datasheet[EB/OL]. [2026-06-16].
<https://www.st.com/en/microcontrollers-microprocessors/stm32f411rc.html>.
- [3] OpenCV team. OpenCV Tutorials: Introduction to OpenCV[EB/OL]. [2026-06-16].
https://docs.opencv.org/5.0/tutorials/introduction/table_of_content_introduction.html.
- [4] ONNX Runtime Developers. ONNX Runtime Documentation[EB/OL].
[2026-06-16]. <https://onnxruntime.ai/docs/>.
- [5] FreeRTOS.org. FreeRTOS Documentation[EB/OL]. [2026-06-16].
<https://www.freertos.org/Documentation/00-Overview>.
- [6] MySQL 开发团队. MySQL 8.0 Reference Manual[EB/OL]. [2026-06-16].
<https://dev.mysql.com/doc/>.
- [7] The Qt Company. Qt Documentation[EB/OL]. [2026-06-16]. <https://doc.qt.io/>.
- [8] 张大头闭环伺服. ZDT_X 系列_V2 步进闭环驱动说明书 Rev1.0[EB/OL].
[2026-06-16]. <https://blog.csdn.net/zhangdatou666/article/details/135709033>.
- [9] 进迭时空 SpacemiT. 手把手教你学 RISC-V Linux 开发[EB/OL]. [2026-06-16].
<https://www.bilibili.com/video/BV1Dh5Lz7EQD/>.
- [10] Kevin_WWW. 【1.2】CubeMX 创建 FreeRTOS 入门示例——Kevin 带你读
《STM32Cube 高效开发教程高级篇》[EB/OL]. [2026-06-16].
<https://www.bilibili.com/video/BV1uj411v7He/>.
- [11] 王维波, 鄢志丹, 王钊. STM32Cube 高效开发教程（高级篇）[M]. 北京: 人民邮电出版社, 2022.